コアJavascript 3

this

Javascriptのすべての変数は特定オブジェクトのプロパティとして、動作する。

ここでの特定オブジェクトは実行コンテキストである。

```
> var a = 3;
<· undefined
> console.log(a);
  console.log(window.a);
  console.log(this.a);
  3                    51706-5b63f2c0f63a454b.js:1
  3                    51706-5b63f2c0f63a454b.js:1
  3                    51706-5b63f2c0f63a454b.js:1
```

そして、thisはグローバル空間にあるのでグローバルオブジェクトになる。

つまり グローバル変数を宣言すると、Javascriptエンジンはグローバルオブジェクトのプロパティに割り当てる。

関数とメソッドの違い

関数は独立的な機能を実行する一方メソッドは自分を呼び出したオブジェクトに関する動作を実行する。

そして、Javascriptは状況についてthisに様々な値を割り当て実装する。

```
var func = function (x) {
    console.log(this, x);
};

func(6);

var obj = {
    method: func
};

obj.method(4);
```

```
                                51706-5b63f2c0f63a454b.js:1
▶ Window {0: Window, 1: Window, window: Window, sel
  f: Window, document: document, name: '', locatio
  n: Location, ···}
6
▶ {method: f} 4                 51706-5b63f2c0f63a454b.js:1
```

画像を見ると関数の変数funcはthisを出力する場合はWindowになる。

メソッドの形obj.methodはmethod: fに表示される。

つまり、thisはWindow, オブジェクトの違いがある。

コンストラクタ関数のthis

関数がコンストラクタ関数として呼び出した場合は内部のthisは新しく作られるオブジェクト自分になる

```
    var car = function(){
        this.name = 'car';
    }
    var robot = function() {
        this.name='robot';
    }
```

```
undefined
```

```
var c = new car();
var r = new robot();
```

```
undefined
```

```
console.log(c, r);
```

```
▶ car {name: 'car'}     51706-5b63f2c0f63a454b.js:1
▶ robot {name: 'robot'}
```

画像を見るとcarにつくれる変数cのthisはcarになると、robotにつくれる変数rのthisはrobotになる。

callメソッド

callメソッドはメソッドを呼び出す関数すぐに実行するメソッドである。

関数をただ実行するとthisはグローバルオブジェクトを参照するが、callメソッドを使う場合はオブジェクトをthisに指定することができる。

```
var func = function (a, b, c) {
    console.log(this, a, b, c);
}

func(1, 2, 3);
func.call({x:1}, 4, 5, 6);
                                51706-5b63f2c0f63a454b.js:1
 ▶ Window {0: Window, 1: Window, window: Window, sel
   f: Window, document: document, name: '', locatio
   n: Location, ···}
1 2 3
 ▶ {x: 1} 4 5 6                 51706-5b63f2c0f63a454b.js:1
```

画像を見るとただの関数に引数を入れて呼び出す場合はthisはグローバルオブジェクト(window)になり、{x:1}を入れてcallメソッドを呼び出すとthisは画像のように変わる。

```
> func({x:1},4, 5, 6);
                                51706-5b63f2c0f63a454b.js:1
 ▶ Window {0: Window, 1: Window, window: Window, sel
   f: Window, document: document, name: '', locatio
   n: Location, ···}
 ▶ {x: 1} 4 5
```

画像を見るとcallメソッドを使わなければオブジェクトを指定することができない。

callメソッドの活用

類似配列オブジェクト(array-like objects)

```
var obj_arr = {
    0:'a',
    1:'b',
    2:'c',
    length:3
}
```

このように配列ではないが配列と似ている形でオブジェクトを作るのができる。

これが類似配列オブジェクトである。

類似配列オブジェクト理解できないので検索しました。

- https://joyhong-91.tistory.com/15

配列ではないので配列に関する関数は使えない。

それで、callメソッドを使って一時的に使用する。

```
Array.prototype.push.call(obj_arr, 'd');
4
console.log(obj_arr);
                    51706-5b63f2c0f63a454b.js:1
▸ {0: 'a', 1: 'b', 2: 'c', 3: 'd', length: 4}
```

bind

bindメソッドは関数にthisを設定することと部分的な関数を設定することを実装する。

```
var func = function (a, b, c) {
    console.log(this, a, b, c);
}
func(1,2,3);
                    51706-5b63f2c0f63a454b.js:1
▸ Window {0: Window, 1: Window, window: Window, sel
  f: Window, document: document, name: '', locatio
  n: Location, …}
1 2 3
```

画像のような関数を

```
var bind_func1 = func.bind({x:1});
undefined
bind_func1(7,3,2);
▸ {x: 1} 7 3 2          51706-5b63f2c0
```

blindメソッドをつかってthisを設定する。

```
> var bind_func2 = func.bind({list:[1,2,3]},5);
<· undefined
```

引数と使う場合

```
bind_func2(4);
▸ {list: Array(3)} 5 4  51706-5b63f2c0f63a45
undefined
```

```
bind_func2(7,1);
▸ {list: Array(3)} 5 7  51706-5b63f2c0f
1
```

部分的な関数を設定することができる。